

# Principles of Programming Languages

Implementation methods

Presented by: Kalyani Vidhate

COEP Technological University, Pune

# Implementation Methods

1. Compilation
2. Interpretation
3. Hybrid Implementation
4. Preprocessors

# Implementation Methods

- **Compiler:** A program that translates an executable program in one language into an executable program in another language
  - During this process of translation if some errors are encountered, then compiler displays them as error message
  - It takes programs such as C, PASCAL, FORTRAN and converts into lower level languages like assembly language
- **Interpreter:** A program that reads an executable program and produces the results of running that program
  - Reading: The interpreter reads each statement or expression in the source code.
  - Execution: The statement is executed immediately, often by converting it to an intermediate form or directly manipulating the runtime environment.

# Implementation Methods

- **Hybrid Implementation:** A hybrid implementation combines both compilation and interpretation to balance speed and flexibility.
- **Bytecode Compilation:** Source code is first compiled into an intermediate bytecode, which is platform-independent.
- **Execution:** The bytecode is then interpreted by a virtual machine (VM) specific to the platform.
- **Cross-platform compatibility and better performance than pure interpretation.** Still slower than fully compiled languages and requires a VM.
- **Examples: Java:** Source code is compiled into Java bytecode, which runs on the Java Virtual Machine (JVM). **C#:** Compiles into Common Intermediate Language (CIL) for execution on the .NET Common Language Runtime (CLR).

# Implementation Methods

**Preprocessors :** A preprocessor processes the source code before it reaches the compiler, performing tasks like macro expansion, file inclusion, and conditional compilation.

## Common Preprocessing Directives:

- **Macro Expansion (`#define`):** Replaces defined symbols with their corresponding values or code.
- **File Inclusion (`#include`):** Includes the contents of a file into the current file.  
Eg. `#include <stdio.h>`
- **Conditional Compilation (`#ifdef`, `#ifndef`, `#endif`):** Compiles code selectively based on conditions.

Simplify coding, make programs more portable, and allow for conditional compilation.

**Examples:** C/C++ preprocessors.

# Programming Environments

1. Integrated Development Environments (IDEs)
2. Command-Line Environments
3. Version Control Systems

# Programming Environments

**1. Integrated Development Environments (IDEs):** A software application that provides comprehensive facilities to programmers for software development. Streamlines development processes, integrates various tools, and improves productivity.

## Key Features:

1. **Source Code Editor:** Syntax highlighting, code completion, and error detection.
2. **Compiler/Interpreter Integration:** Ability to compile or interpret code directly within the environment.
3. **Debugger:** Tools for testing and debugging code.
4. **Build Automation Tools:** Automate the process of compiling and linking code.
5. **Version Control Integration:** Manage code versions and collaborate with others.

## Examples:

- **Visual Studio:** Popular for .NET development.
- **Eclipse:** Widely used for Java development.
- **IntelliJ IDEA:** Known for its powerful features in Java and Kotlin development.
- **PyCharm:** Specializes in Python development.

Suggest More !!

# Programming Environments

**2. Command-Line Environments:** Tools and compilers used via command-line interfaces (CLI) to compile and manage code. Lightweight, flexible, and powerful for automating tasks.

## Examples:

- **GCC (GNU Compiler Collection):** Widely used for compiling C, C++, and other languages.

```
# Open your terminal or command prompt

# Navigate to the directory where your hello.c file is located
cd path/to/your/directory

# Compile the C program using GCC
gcc hello.c -o hello

# Run the compiled program
./hello
```

- **GDB (GNU Debugger):** A powerful tool for debugging C and C++ programs.
- **Make:** Automates the build process using Makefiles.
- **Gradle/Maven:** Build automation tools for Java projects.



# Programming Environments

**3. Version Control Systems:** Tools that track and manage changes to the source code over time. Facilitates collaboration, tracks history, and allows rollback of changes.

## **Features:**

1. **Commit:** Save changes to the repository with a message describing the changes.
2. **Branching:** Create separate lines of development within the project.
3. **Merging:** Combine changes from different branches.
4. **Pull Requests:** Request to merge changes from one branch to another, often used for code reviews.

## **Examples:**

- **Git:** A distributed version control system, used with platforms like GitHub, GitLab.
- **SVN (Subversion):** A centralized version control system.

# Major Programming Languages

1. Pseudocodes
2. Assembly Language
3. C Language
4. C++ Language
5. FORTRAN
6. Java

# Major Programming Languages

## 1. Pseudocodes :

- Pseudocode is a plain language description of the steps in an algorithm, used as an intermediary between the initial problem statement and the final code implementation.
- Facilitates the understanding and communication of algorithms without the need for specific syntax.
- Informal and intuitive, no strict syntax, resembles structured English.

```
IF temperature > 100 THEN  
    PRINT "Boiling"  
ELSE  
    PRINT "Not Boiling"  
ENDIF
```

Why ??

# Major Programming Languages

## 2. Assembly Language:

- Assembly language is a low-level programming language that is one step above machine code and is specific to a computer architecture.
- Uses mnemonic codes to represent machine-level instructions (e.g., **MOV**, **ADD**). Directly manipulates hardware resources like CPU registers and memory.
- Initially used for writing system software, device drivers, and performance-critical applications. Modern usage is mostly in embedded systems, bootloaders, and hardware drivers.
- Highly efficient and fast, as it directly translates to machine code. Complex, hardware-specific, and not portable across different platforms.

```
MOV AX, 1      ; Load the value 1 into register AX
ADD AX, BX     ; Add the value in register BX to AX
MOV [1000h], AX ; Store the result in memory at address 1000h
```

# Major Programming Languages

## 3. C Language:

- Developed in the early 1970s by Dennis Ritchie at Bell Labs for system programming, especially for writing operating systems (e.g., UNIX).
- Low-level access to memory, a simple and efficient language, structured programming paradigm.
- Influential in the development of later languages like C++, Java, and Python. Widely used in systems programming, embedded systems, and for developing performance-critical applications.

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;
    int sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

# Major Programming Languages

## 4. C++ Language:

- Created by Bjarne Stroustrup in the 1980s as an extension of C, adding object-oriented programming features.
- Supports both procedural and object-oriented programming paradigms.
- Introduces **classes, inheritance, polymorphism, templates, and exception handling.**
- Commonly used for developing complex software systems like game engines, GUI applications, and real-time simulations.

```
#include <iostream>
using namespace std;

class Rectangle {
    int width, height;
public:
    Rectangle(int w, int h) : width(w), height(h) {}
    int area() { return width * height; }
};

int main() {
    Rectangle rect(10, 5);
    cout << "Area: " << rect.area() << endl;
    return 0;
}
```

# Major Programming Languages

## 5. FORTRAN:

- Developed by IBM in the 1950s, FORTRAN (FORMula TRANslation) is one of the earliest high-level programming languages, designed for scientific and engineering calculations.
- Strong support for numerical computation and array manipulation, easy to optimize for performance.
- Still widely used in scientific computing, particularly in areas like physics, chemistry, and engineering.

```
PROGRAM Hello  
  PRINT *, 'Hello, world!'  
END PROGRAM Hello
```

# Major Programming Languages

## 6. Java:

- Developed by Sun Microsystems in the mid-1990s, Java was designed to be platform-independent, object-oriented, and secure.
- **Write Once, Run Anywhere (WORA):** Java code is compiled into bytecode, which runs on the Java Virtual Machine (JVM).
- Strong memory management (garbage collection), exception handling, and built-in security features.
- Widely used for web development, enterprise applications, and Android app development.

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```